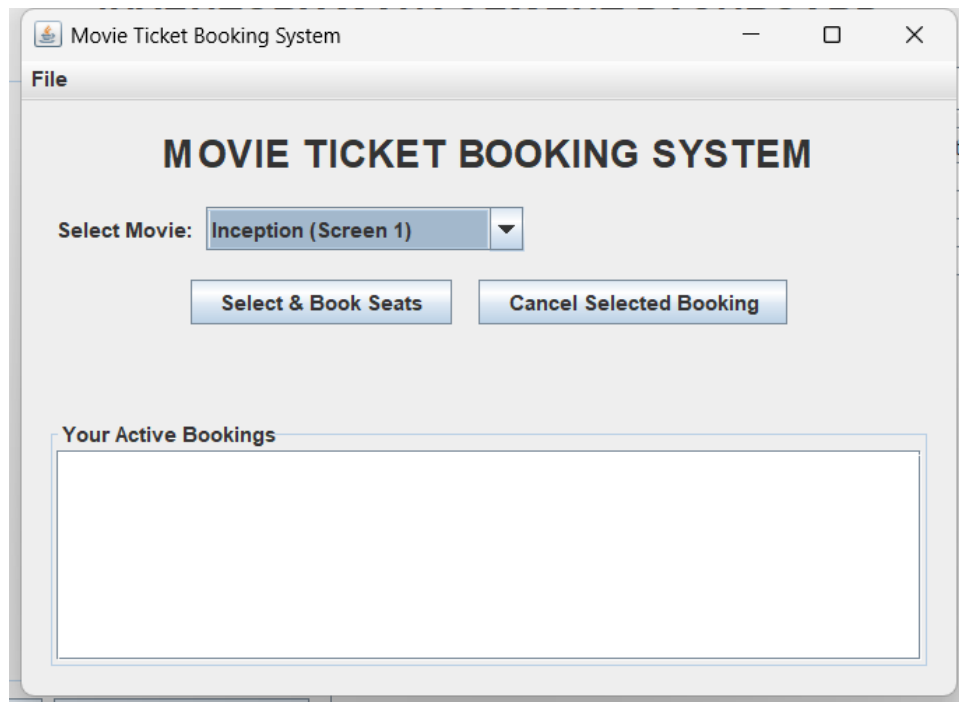


MOVIE TICKET BOOKING SYSTEM

PROGRAM OUTPUT SCREENSHOT



SOURCE CODE

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class TicketBookingSystem extends JFrame {
    private final JComboBox<String> movieComboBox;
    private final DefaultListModel<String> bookingListModel;
    private final JList<String> bookingJList;
    private final Map<String, boolean[]> seatAvailabilityMap = new HashMap<>();
    private final List<Booking> bookings = new ArrayList<>();

    private final String[] movies = {"Inception (Screen 1)", "Interstellar (Screen 2)",
    "The Dark Knight (Screen 3)"};
    private static final int TOTAL_SEATS = 30;

    public TicketBookingSystem() {
        setTitle("Movie Ticket Booking System");
        setSize(550, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        for (String movie : movies) {
            seatAvailabilityMap.put(movie, new boolean[TOTAL_SEATS]);
        }

        JPanel mainPanel = new JPanel(new BorderLayout(12, 12));
        mainPanel.setBorder(new EmptyBorder(15, 15, 15, 15));

        JLabel titleLabel = new JLabel("MOVIE TICKET BOOKING SYSTEM",
        SwingConstants.CENTER);
        titleLabel.setFont(new Font("SansSerif", Font.BOLD, 22));
        mainPanel.add(titleLabel, BorderLayout.NORTH);

        JPanel centerPanel = new JPanel(new GridLayout(3, 1, 10, 10));

        JPanel selectMoviePanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 5, 5));
        selectMoviePanel.add(new JLabel("Select Movie: "));
        movieComboBox = new JComboBox<>(movies);
        selectMoviePanel.add(movieComboBox);
        centerPanel.add(selectMoviePanel);

        JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 15, 5));
        JButton bookSeatsBtn = new JButton("Select & Book Seats");
        JButton cancelBookingBtn = new JButton("Cancel Selected Booking");
        buttonPanel.add(bookSeatsBtn);
        buttonPanel.add(cancelBookingBtn);
        centerPanel.add(buttonPanel);

        mainPanel.add(centerPanel, BorderLayout.CENTER);

        JPanel southPanel = new JPanel(new BorderLayout(5, 5));
        southPanel.setBorder(BorderFactory.createTitledBorder("Your Active Bookings"));
        bookingListModel = new DefaultListModel<>();
        bookingJList = new JList<>(bookingListModel);
        JScrollPane scrollPane = new JScrollPane(bookingJList);
        scrollPane.setPreferredSize(new Dimension(500, 120));
```

```

        southPanel.add(scrollPane, BorderLayout.CENTER);
        mainPanel.add(southPanel, BorderLayout.SOUTH);

        createMenuBar();

        bookSeatsBtn.addActionListener(e -> openSeatSelectionWindow());
        cancelBookingBtn.addActionListener(e -> cancelSelectedBooking());

        add(mainPanel);
        setVisible(true);
    }

    private void createMenuBar() {
        JMenuBar menuBar = new JMenuBar();
        JMenu fileMenu = new JMenu("File");
        JMenuItem exitItem = new JMenuItem("Exit");
        exitItem.addActionListener(e -> System.exit(0));
        fileMenu.add(exitItem);
        menuBar.add(fileMenu);
        setJMenuBar(menuBar);
    }

    private void openSeatSelectionWindow() {
        String selectedMovie = (String) movieComboBox.getSelectedItem();
        if (selectedMovie == null) return;
        new SeatSelectionWindow(selectedMovie);
    }

    private void cancelSelectedBooking() {
        int selectedIndex = bookingJList.getSelectedIndex();
        if (selectedIndex < 0) {
            JOptionPane.showMessageDialog(this, "Please select an active booking from the
list to cancel.", "Warning", JOptionPane.WARNING_MESSAGE);
            return;
        }

        int choice = JOptionPane.showConfirmDialog(this, "Are you sure you want to cancel
the selected booking?", "Confirm Cancellation", JOptionPane.YES_NO_OPTION);
        if (choice == JOptionPane.YES_OPTION) {
            Booking b = bookings.get(selectedIndex);
            boolean[] seats = seatAvailabilityMap.get(b.movie);
            for (int seatNum : b.bookedSeats) {
                seats[seatNum - 1] = false;
            }
            bookings.remove(selectedIndex);
            refreshBookingList();
            JOptionPane.showMessageDialog(this, "Booking cancelled successfully. Seats
released.", "Success", JOptionPane.INFORMATION_MESSAGE);
        }
    }

    private void refreshBookingList() {
        bookingListModel.clear();
        for (Booking b : bookings) {
            bookingListModel.addElement(b.toString());
        }
    }

    class SeatSelectionWindow extends JFrame {
        private final String movie;
        private final JToggleButton[] seatButtons = new JToggleButton[TOTAL_SEATS];
        private final JTextField customerNameField = new JTextField(15);
        private final JTextField phoneField = new JTextField(10);
    }

```

```

SeatSelectionWindow(String movie) {
    this.movie = movie;
    setTitle("Select Seats - " + movie);
    setSize(450, 450);
    setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
    setLocationRelativeTo(TicketBookingSystem.this);
    setLayout(new BorderLayout(10, 10));

    JPanel topPanel = new JPanel(new GridLayout(2, 2, 5, 5));
    topPanel.setBorder(new EmptyBorder(10, 10, 5, 10));
    topPanel.add(new JLabel("Customer Name:"));
    topPanel.add(customerNameField);
    topPanel.add(new JLabel("Phone Number:"));
    topPanel.add(phoneField);
    add(topPanel, BorderLayout.NORTH);

    JPanel gridPanel = new JPanel(new GridLayout(5, 6, 8, 8));
    gridPanel.setBorder(BorderFactory.createTitledBorder("Screen is here (Top)"));
    boolean[] availability = seatAvailabilityMap.get(movie);

    for (int i = 0; i < TOTAL_SEATS; i++) {
        int seatNum = i + 1;
        seatButtons[i] = new JToggleButton(String.valueOf(seatNum));
        if (availability[i]) {
            seatButtons[i].setBackground(Color.RED);
            seatButtons[i].setForeground(Color.WHITE);
            seatButtons[i].setEnabled(false);
        } else {
            seatButtons[i].setBackground(Color.GREEN);
        }
        gridPanel.add(seatButtons[i]);
    }
    add(gridPanel, BorderLayout.CENTER);

    JButton confirmBtn = new JButton("Confirm Booking");
    confirmBtn.addActionListener(this::confirmBooking);
    JPanel southPanel = new JPanel(new FlowLayout(FlowLayout.CENTER));
    southPanel.add(confirmBtn);
    add(southPanel, BorderLayout.SOUTH);

    setVisible(true);
}

private void confirmBooking(ActionEvent e) {
    String name = customerNameField.getText().trim();
    String phone = phoneField.getText().trim();

    if (name.isEmpty() || phone.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Customer Name and Phone Number are
required.", "Validation Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
    if (!phone.matches("\\d{10}")) {
        JOptionPane.showMessageDialog(this, "Phone number must be a 10-digit
numeric value.", "Validation Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    List<Integer> selectedSeats = new ArrayList<>();
    for (int i = 0; i < TOTAL_SEATS; i++) {
        if (seatButtons[i].isSelected() && seatButtons[i].isEnabled()) {
            selectedSeats.add(i + 1);
        }
    }
}

```

```

        if (selectedSeats.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Please select at least one seat.",
"Validation Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        boolean[] availability = seatAvailabilityMap.get(movie);
        for (int seatNum : selectedSeats) {
            availability[seatNum - 1] = true;
        }

        Booking booking = new Booking(name, phone, movie, selectedSeats);
        bookings.add(booking);
        refreshBookingList();

        JOptionPane.showMessageDialog(this, "Booking confirmed successfully for seats:
" + selectedSeats, "Success", JOptionPane.INFORMATION_MESSAGE);
        this.dispose();
    }
}

static class Booking {
    String customerName;
    String phone;
    String movie;
    List<Integer> bookedSeats;

    Booking(String customerName, String phone, String movie, List<Integer> bookedSeats)
    {
        this.customerName = customerName;
        this.phone = phone;
        this.movie = movie;
        this.bookedSeats = bookedSeats;
    }

    @Override
    public String toString() {
        return customerName + " (" + phone + ") - " + movie + " | Seats: " +
bookedSeats.toString();
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(TicketBookingSystem::new);
}
}

```